

Agile+MSA_과제_8반_최규원

Chapter 1. Agile·Scrum 이해

1. 왜 필요한가

Waterfall처럼 계획 다 세우고 한 번에 만드는 게 아니라, 짧은 주기(Sprint)로 만들고 → 보여주고 → 고치는 걸 반복하는 방식이라는 걸 이번에 이해했다. "그냥 빨리 대충 만드는 거"인 줄 알았는데, 오히려 매 Sprint마다 점검하는 절차가 더 촘촘한 느낌이었다.

2. 핵심 용어

- **Sprint**: 정해진 기간(보통 1개월 이하) 안에 쓸 수 있는 결과물을 만드는 반복 단위
- **Product Backlog**: 제품에 필요한 일 전체 목록. 계속 바뀔 수 있음
- **Sprint Backlog**: 그중 이번 Sprint에서 할 것만 뽑아놓은 것
- **User Story**: "~로서 ~하기 위해 ~하고 싶다" 형식. 근데 이거 Scrum 공식 산출물은 아니라고 함
- **DoD**: "이 정도면 끝"이라고 팀이 합의한 기준

3. Scrum 4대 이벤트

이벤트	하는 일	누가
Planning	이번 Sprint 목표·할 일 정하기	팀 전체
Daily Scrum	매일 15분, 진행상황 체크	개발자들
Review	결과물 보여주고 피드백 받기	팀 + 이해관계자
Retro	일하는 방식 자체를 돌아보기	팀 전체

4. 헷갈렸던 것

- Product Backlog는 제품 전체 할 일, Sprint Backlog는 이번 Sprint 것만 → 처음엔 구분이 안 갔다
- Review는 "뭘 만들었는지"를 보는 거고 Retro는 "어떻게 일했는지"를 보는 거라 목적이 다르다. 둘이 비슷한 건 줄 알았다.

Chapter 2. MSA 시스템 구조 및 설계 이해

1. 프로젝트 구조 개요

본 실습은 Spring Boot 기반 마이크로서비스(Eureka, Auth Server, User/Course/Enrollment/Payment Service) + FastAPI 기반 추천 서비스로 구성된 온라인 강의 플랫폼 코드 템플릿을 로컬에서 실행하며 구조를 파악하는 것으로 시작했다.

```
msa-lecture/  
├─ docker-compose.yml  
├─ eureka-server/  
├─ user-service/  
├─ course-service/  
├─ enrollment-service/  
├─ payment-service/  
├─ recommend-service/ ← 애만 Python/FastAPI  
├─ vue-frontend/  
└─ ...
```

Auth Server랑 Gateway는 소스가 따로 없고 이미 빌드된 이미지로만 실행되는 구조였다. 하나로 안 뭉쳐있고 역할별로 쪼개져 있다는 게 MSA 구조인 걸 확인했다.

2. 각 컴포넌트의 역할

- **Eureka**: 서비스들 위치 등록/조회하는 명부 같은 것
- **API Gateway**: 요청 제일 먼저 받는 곳. 인증 확인하고 알맞은 서비스로 넘김
- **Auth Server**: 로그인, 토큰 발급
- **User / Course / Enrollment / Payment**: 이름 그대로 각자 도메인 데이터 처리
- **Recommend**: 수강 이력 보고 추천 (AI 아니고 규칙 기반이라는 게 의외였음)
- **Kafka**: 결제완료 같은 이벤트 비동기로 전달
- **MariaDB**: 전체 데이터 저장

3. 실행 환경 구성

전체 10개 컨테이너(MariaDB, Kafka, Eureka, Auth Server, Gateway + 비즈니스 서비스 5개) 구성.

서비스	포트	비고
API Gateway	8080	전체 진입점
User / Course / Enrollment / Payment	8081~8084	서비스별 Swagger UI로 확인
Recommend	8085	FastAPI라 /docs로 확인
Auth Server	9000	로그인 · OAuth2
Eureka	8761	서비스 등록 대시보드
Kafka	9092	이벤트 전달
MariaDB	3379 → 3306	로컬에선 3379로 접근

실행 명령어:

```
cat msa-lecture-images.part* > msa-lecture-images.tar.gz
docker load -i msa-lecture-images.tar.gz
docker load -i infra-images.tar
docker compose up -d --no-build --pull never
```

`docker compose ps` 찍어보니 10개 다 떠 있었고, 8081~8085 Swagger/문서 페이지도 다 정상 응답했다.

4. 시스템 요청 처리 흐름

1. 사용자 로그인 → Auth Server가 인증 후 토큰 발급
2. 강의 조회 → Course Service가 MariaDB에서 조회해 반환
3. 수강신청 → Enrollment Service가 강의 존재 여부 확인 후 `PENDING` 상태로 저장
4. 결제 → Payment Service가 UUID 거래번호로 가짜 결제 처리 후 `COMPLETED` 로 변경, Kafka로 `payment.completed` 발행
5. Enrollment Service가 이벤트를 받아 수강 상태를 `ACTIVE` 로 변경 → `enrollment.completed` 재발행
6. Recommend Service가 이를 받아 추천 목록 갱신

```
로그인(Auth Server, 토큰 발급)
→ 강의 조회(Course Service)
→ 수강신청(Enrollment, PENDING 저장)
→ 결제(Payment, UUID로 가짜 결제 → COMPLETED)
→ Kafka: payment.completed → Enrollment가 받아서 ACTIVE로 변경
→ Kafka: enrollment.completed → Recommend가 추천 갱신
```

결제가 실제 PG 연동이 아니라 UUID로 바로 COMPLETED 처리하는 거라 좀 신기했다. 결제 → 수강활성화가 REST 직접호출이 아니라 굳이 Kafka를 거치는데, 아마 Payment가 Enrollment 상태 바뀔 때까지 안 기다려도 되게 하려는 거 같다 (확실히 않아서 더 찾아볼 것).

Chapter 3. 트러블슈팅 사례 정리

1. compose 파일 교체 오류

-  `docker-compose.local.yml` 을 `docker-compose.yml` 로 바꾸려다 → `mv: rename docker-compose.local.yml to docker-compose.yml: No such file or directory`
- 원인: 애초에 로컬용 compose 파일이 폴더에 없었음
-  `find` 로 위치 확인 후 파일 다시 받아서 배치하여 해결

2. Docker 데몬 미실행

-  `docker load` 실행 시 → `failed to connect to the docker API ... docker.sock: no such file or directory`
- 원인: Docker Desktop 자체가 꺼져 있었음
-  `open -a Docker` 로 실행 후 `docker info` 로 확인, 재시도하여 해결

3. compose 파일 부재 상태에서 실행 시도

-  `no configuration file provided: not found`
- 원인: 파일 이름을 옮기는 과정에서 순서가 꼬여 기본 compose 파일이 아직 없었음
-  파일 배치 후 `docker compose config` 로 먼저 검증, 이후 실행하여 해결

4. 최종 실행

- 파일과 Docker 엔진이 모두 준비된 상태에서 이미지 로드 후 `docker compose up -d --no-build --pull never` 로 실행 → 별문제 없이 10개 컨테이너 모두 정상 기동

✓ 트러블슈팅 요약

- 에러 메시지만 보고 바로 명령어 탓하지 말고, 파일 존재 여부·Docker 엔진 상태부터 순서대로 확인해야 한다는 걸 오늘 세 번이나 겪으며 배웠다
- Agile/Scrum은 계획 없이 빨리 만드는 게 아니라 짧은 주기로 점진·개선하는 방식이라는 걸 이해했다
- MSA는 Gateway/Eureka/Auth/비즈니스서비스/Kafka가 역할을 나눠서 따로 도는 구조라는 걸 직접 확인했다.